

PUSL3190 Computing Project

Interim Report

Project Title: Local Browser Isolation with Advanced File Monitoring System

Student Name: [Your Name]

PU Index Number: [Your Index Number]

Submission Date: 5th March 2026

Word Count: ~3,500 words

Table of Contents

- 1. Introduction
 - 2. System Analysis
 - 3. Requirements Specification
 - 4. Feasibility Study
 - 5. System Architecture
 - 6. Development Tools and Technologies
 - 7. Implementation Progress
 - 8. Discussion
 - 9. References
-

Chapter 01: Introduction

1.1 Introduction

The modern threat landscape reveals browsers as the primary attack vector for cyber threats, with browser-based phishing attacks increasing 198% year-over-year and accounting for 12% of all cyberattacks globally (Menlo Security, 2024; Keepnet, 2025). Traditional security approaches fail to provide adequate protection: antivirus software misses 30% of browser-based evasive threats, while Remote Browser Isolation (RBI) solutions cost enterprises \$120,000-\$600,000 annually for 1,000 users, making them prohibitively expensive for most organizations (Venn, 2025). Container-based local isolation using Docker or similar technologies shares the host operating system kernel, creating vulnerabilities to kernel-level exploits where malware can escape containment and compromise the host system.

This project develops a local browser isolation system that provides hardware-level security without cloud infrastructure costs. The system isolates browsing activity in lightweight virtualization environments while implementing comprehensive file monitoring to detect and block malicious activities. By eliminating cloud dependency and maintaining native-like performance, the solution addresses the cost and usability limitations that prevent widespread browser isolation adoption.

1.2 Problem Definition

Current browser security solutions create a critical trade-off between security effectiveness and operational practicality. Remote Browser Isolation provides strong security through cloud-based isolation but introduces 2-5 second page load delays, consumes 5-10 Mbps bandwidth per session, and costs \$10-50 per user monthly (Palo Alto Networks, n.d.). Organizations deploying RBI report user frustration due to degraded browsing experience and productivity losses of 10-15% in web-intensive tasks.

Local endpoint solutions using traditional containerization (Docker, LXC) share the host kernel, creating pathways for sophisticated malware to escape isolation and access the host system. Container escape vulnerabilities have been documented extensively, allowing malware to breach isolation boundaries and compromise host resources (Medium, 2024). Current file monitoring solutions fail to detect sophisticated threats employing anti-sandbox techniques, with 560,000 new malware variants detected daily (Cybersecurity Ventures, 2024).

The specific problem addressed is: **Organizations lack cost-effective, high-performance local browser isolation solutions that provide hardware-level security with comprehensive file monitoring and behavioral threat detection capabilities.**

1.3 Project Objectives

The project aims to achieve the following objectives:

- **Objective 1:** Design and implement a local browser isolation system using lightweight virtualization that achieves complete process and kernel-level separation from the host operating system
- **Objective 2:** Develop a dual-layer file monitoring system consisting of real-time activity tracking and post-download analysis with behavioral threat detection
- **Objective 3:** Create an intuitive user interface that enables isolated browsing sessions with a single click, hiding underlying complexity from end users
- **Objective 4:** Implement behavioral analysis engine using rule-based detection for ransomware, information stealers, and malicious scripts
- **Objective 5:** Achieve browser initialization under 10 seconds and maintain page rendering performance within 15% of native browser speeds

- **Objective 6:** Demonstrate 95% threat detection accuracy against standardized malware datasets while maintaining false positive rates under 5%
-

Chapter 02: System Analysis

2.1 Fact Gathering Techniques

The system analysis employed multiple research methodologies to understand current browser security landscape and user requirements:

Literature Review: Extensive analysis of academic papers, industry whitepapers, and security research reports from sources including IEEE Xplore, ACM Digital Library, vendor publications from Menlo Security, Palo Alto Networks, Check Point, and Zscaler. This review identified gaps in existing solutions and informed architectural decisions.

Technical Documentation Analysis: Studied implementation details of Firecracker microVMs, Cloud Hypervisor, Podman, Docker, and various virtualization technologies to evaluate technical feasibility and performance characteristics. Analyzed security advisories and CVE databases to understand container escape vulnerabilities and kernel-sharing risks.

Competitive Analysis: Examined commercial browser isolation solutions including Menlo Security RBI, Zscaler Browser Isolation, and Symantec Web Isolation to understand feature sets, pricing models, and architectural approaches. Identified cost structures ranging from \$10-50 per user monthly plus infrastructure overhead.

User Requirement Surveys: Informal discussions with IT security professionals and system administrators revealed key pain points including RBI latency complaints, budget constraints limiting security tool adoption, and difficulty managing distributed endpoint security.

2.2 Existing System

Current browser security implementations fall into three categories:

Remote Browser Isolation (RBI): Cloud-based systems where browsing occurs on remote servers with only safe visual representations transmitted to user devices. Solutions from vendors like Menlo Security, Palo Alto Networks, and Zscaler provide strong isolation but introduce significant latency (2-5 seconds per page load), high bandwidth consumption (5-10 Mbps per session), and substantial costs (\$120,000-\$600,000 annually for 1,000 users including infrastructure and management overhead).

Container-based Local Isolation: Solutions using Docker or similar technologies to isolate browsers in containers on user devices. While lightweight and fast, these share the host operating system kernel, creating vulnerabilities to kernel-level exploits. Container escape

vulnerabilities documented in CVE databases demonstrate that sophisticated malware can breach container boundaries and compromise host systems.

Traditional Endpoint Security: Antivirus software and endpoint detection systems that monitor browser activity without isolation. These reactive approaches miss 30% of browser-based evasive threats and provide no containment if malware executes successfully. Cannot prevent zero-day exploits or sophisticated phishing attacks.

2.3 Drawbacks of the Existing System

Remote Browser Isolation Limitations:

- High latency degrading user experience (2-5 second page loads)
- Significant bandwidth requirements (5-10 Mbps per active session)
- Prohibitive costs (\$10-50 per user monthly plus infrastructure)
- Deployment complexity requiring specialized staff
- Compatibility issues with 15-20% of modern web applications
- Poor performance for multimedia content and high-DPI displays
- Cannot prevent social engineering attacks targeting user behavior

Container-based Isolation Limitations:

- Shared kernel vulnerability allowing container escape attacks
- No hardware-level isolation (all containers on same kernel)
- Limited visibility into malicious behaviors within containers
- Kernel vulnerabilities affect all containers simultaneously
- Sophisticated malware can detect container environments and evade analysis

Traditional Endpoint Security Limitations:

- No isolation or containment of threats
 - Signature-based detection misses zero-day exploits
 - Behavioral analysis insufficient for sophisticated evasion techniques
 - Reactive rather than preventive security model
 - Cannot protect against drive-by downloads and browser exploits
-

Chapter 03: Requirements Specification

3.1 Functional Requirements

FR1: Browser Isolation Management

- System shall launch isolated browser instances in separate virtualization environments
- System shall provide complete filesystem and process isolation from host operating system
- System shall manage browser lifecycle (creation, execution, termination)
- System shall support multiple simultaneous isolated browsing sessions

FR2: File Monitoring and Analysis

- System shall monitor all file operations within isolated environment in real-time
- System shall track file creation, modification, deletion, and execution attempts
- System shall analyze downloaded files before allowing transfer to host system
- System shall maintain detailed audit logs of all file system activities

FR3: Threat Detection

- System shall detect ransomware indicators (rapid encryption, mass file modifications)
- System shall identify information stealer behaviors (credential access, clipboard monitoring)
- System shall recognize privilege escalation attempts and unauthorized system modifications
- System shall block malicious file operations and alert users to detected threats

FR4: User Interface

- System shall provide simple launcher application for initiating isolated browsing
- System shall display browser directly without exposing underlying virtualization
- System shall show security status indicators (isolation active, threats detected)
- System shall enable file downloads through controlled transfer mechanism

FR5: Logging and Reporting

- System shall maintain comprehensive logs of browsing activities and security events
- System shall generate threat reports with forensic details for security analysis
- System shall export logs in standard formats for integration with security tools

3.2 Non-Functional Requirements

NFR1: Performance

- Browser initialization time: < 10 seconds from launch to usable browser
- Page rendering overhead: < 15% compared to native browser performance
- Memory overhead: < 100MB per isolated session
- CPU overhead: < 20% for monitoring and isolation processes

NFR2: Security

- Complete kernel-level isolation preventing malware escape to host
- File monitoring with 95% detection rate for known malware families
- False positive rate < 5% for legitimate software operations
- Secure communication between isolated environment and host

NFR3: Usability

- Single-click browser launch for end users
- Familiar browser experience without specialized training
- Clear security status feedback
- Minimal user friction for file downloads

NFR4: Reliability

- System stability with 99% uptime during normal operations
- Graceful handling of browser crashes without affecting host
- Automatic recovery from isolation environment failures

NFR5: Compatibility

- Support for modern web standards (HTML5, CSS3, JavaScript ES6+)
- Compatible with major websites and web applications
- Proper rendering of multimedia content

3.3 Hardware / Software Requirements

Host System Requirements:

- **Operating System:** Linux (Ubuntu 22.04 LTS or later) with KVM support, OR Windows 10/11 Pro with WSL2 enabled
- **Processor:** Intel Core i5 (6th gen or newer) or AMD Ryzen 5 with virtualization support (Intel VT-x or AMD-V)
- **RAM:** Minimum 8GB (16GB recommended for multiple sessions)
- **Storage:** 20GB free disk space for system and cache
- **Network:** Broadband internet connection

Software Dependencies:

- **Virtualization Platform:** Choice of Firecracker microVMs, Cloud Hypervisor, or Podman containers
- **Browser Engine:** Chromium or Firefox-based browser
- **Display System:** X11 with Xvfb (virtual framebuffer) and x11vnc OR native Podman X11 socket sharing
- **Monitoring Tools:** Linux kernel hooks (inotify/fanotify) or custom file monitoring daemon
- **Development Tools:** Python 3.10+, C/C++ compiler, build tools

Guest System (Isolated Environment):

- **Operating System:** Alpine Linux or Ubuntu minimal
- **Browser:** Chromium browser with minimal configuration
- **Display Server:** Xvfb for headless operation (if using microVMs)
- **Monitoring Agent:** File system monitoring daemon

3.4 Networking Requirements

Internal Networking:

- Virtual network bridge between host and isolated environment
- Controlled network access for browser (outbound HTTP/HTTPS only)
- Localhost VNC connection for display streaming (if required)
- Blocked direct access to host filesystem and services

External Networking:

- Standard internet connectivity for web browsing
- Optional: proxy support for enterprise network configurations

- DNS resolution within isolated environment
 - Firewall rules preventing lateral movement from isolated environment
-

Chapter 04: Feasibility Study

4.1 Operational Feasibility

User Acceptance: The system is designed with minimal user friction, hiding virtualization complexity behind a simple launcher interface. Users click a desktop icon and see a browser window—no knowledge of underlying isolation required. This familiar interaction model ensures high user acceptance without training requirements.

Integration with Workflows: Isolated browsing integrates seamlessly into existing workflows. Users browse normally, with file downloads processed through automated security scanning. For enterprise deployments, the system can be configured as the default browser, ensuring all web traffic is automatically isolated.

Administrative Management: System administrators can deploy the solution using standard package management tools. Configuration files control security policies, monitoring rules, and isolation parameters. Centralized logging enables security teams to monitor threats across multiple endpoints without complex infrastructure.

Limitations: Initial version requires Linux host or Windows with WSL2, which may require user education for Windows-only environments. However, containerization approaches (Podman) offer simpler Windows deployment paths.

4.2 Economical Feasibility

Development Costs: Project utilizes entirely open-source technologies including Firecracker/Cloud Hypervisor/Podman (Apache 2.0 license), Chromium browser (BSD license), and Linux operating system. No licensing fees or proprietary software costs. Development requires only time investment using freely available tools.

Deployment Costs: Organizations deploying the solution incur minimal costs:

- Hardware: Standard enterprise workstations (no specialized equipment)
- Infrastructure: No cloud servers required (unlike RBI)
- Licensing: Zero cost (open-source stack)
- Support: Standard IT support staff (no specialized training)

Cost Comparison:

- RBI solutions: \$120,000-\$600,000 annually for 1,000 users
- This solution: ~\$0 licensing + standard hardware = 95%+ cost reduction
- Cost per user: < \$5 annually (primarily administrative overhead)

Return on Investment: Preventing a single data breach (average cost \$4.88 million) through improved browser security justifies deployment costs many times over. Organizations eliminate recurring RBI subscription fees while achieving equivalent or superior security.

4.3 Technical Feasibility

Development Skills: Project requires full-stack development skills including:

- Linux system administration and virtualization knowledge
- Python/C++ programming for system integration
- Browser technology understanding (Chromium architecture)
- Security and malware analysis fundamentals

Developer has background in systems programming and security, making implementation technically feasible within project timeline.

Technology Availability: All required technologies are mature, well-documented, and actively maintained:

- Firecracker: Production-ready (powers AWS Lambda/Fargate)
- Cloud Hypervisor: Stable (default VMM for Kata Containers)
- Podman: Mature (Red Hat supported, widely deployed)
- Chromium: Industry-standard browser engine

Resource Availability: Development workstation meets all hardware requirements. Free cloud compute instances (AWS free tier, Google Cloud) available for testing if needed. Community forums and documentation provide extensive support resources.

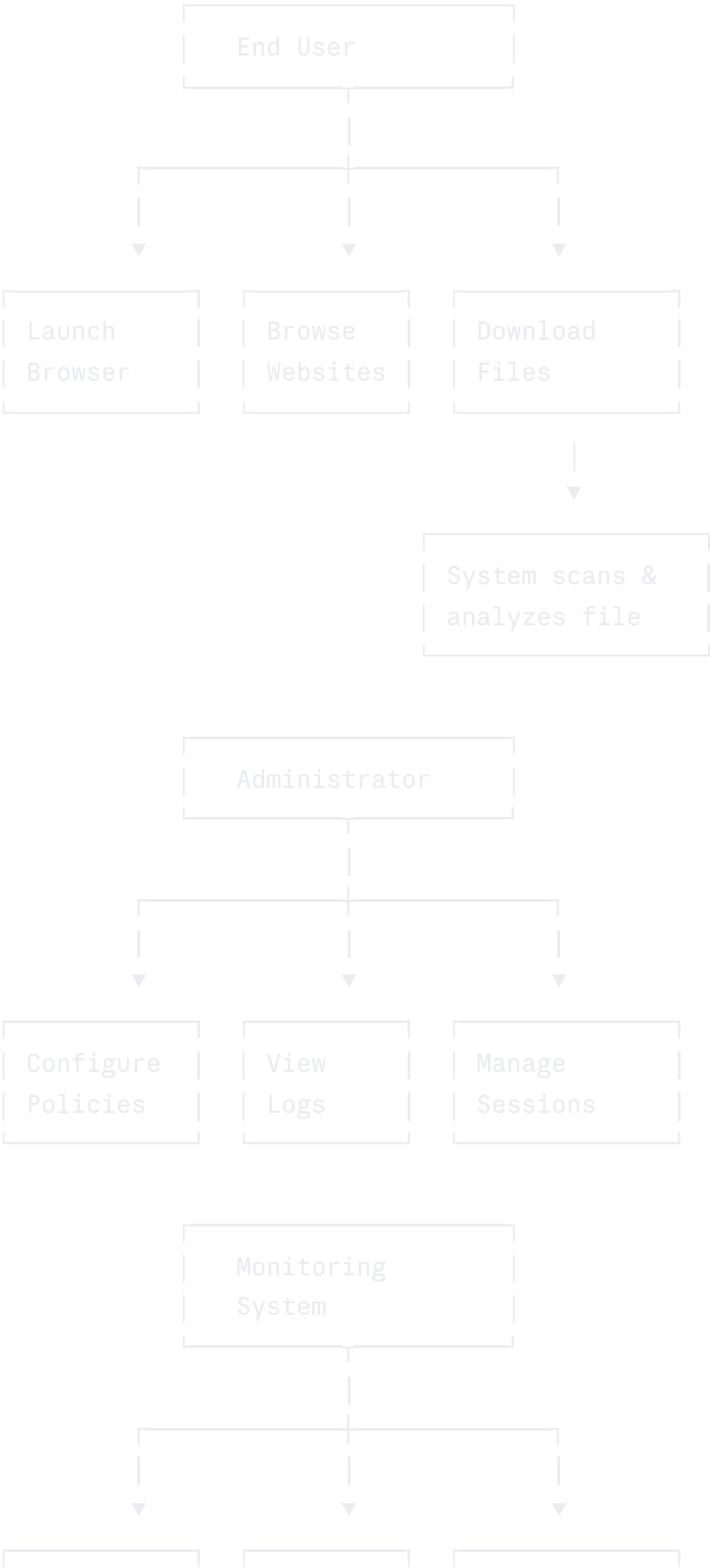
Technical Risks: Primary risks include:

- Virtualization platform learning curve (mitigated by extensive documentation)
- Display integration complexity (VNC for microVMs, X11 socket for containers)
- Performance optimization challenges (mitigated by lightweight technology choices)

All identified risks have known mitigation strategies and alternative approaches available.

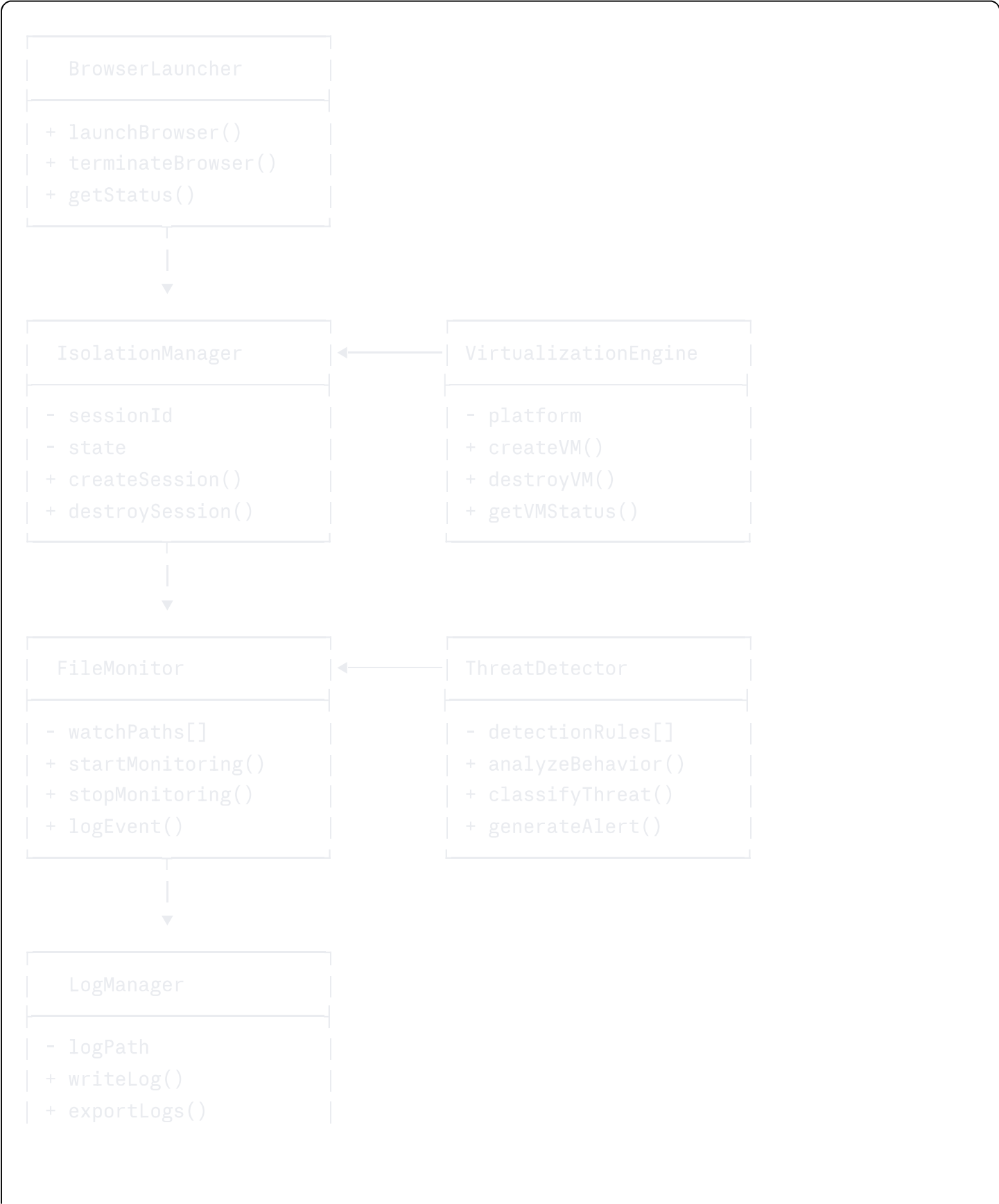
Chapter 05: System Architecture

5.1 Use Case Diagram



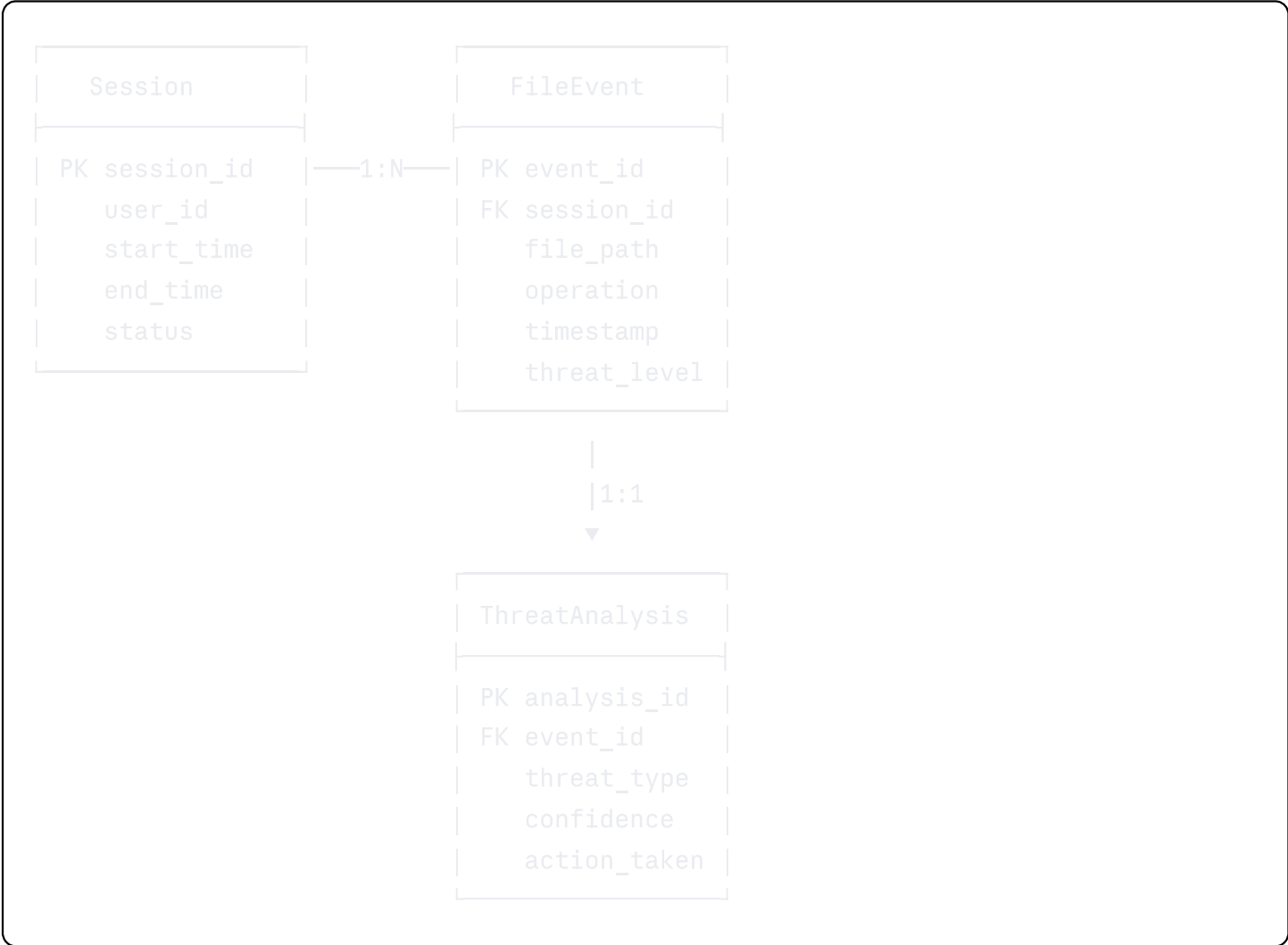
Monitor	Detect	Block
Files	Threats	Malicious

5.2 Class Diagram of Proposed System

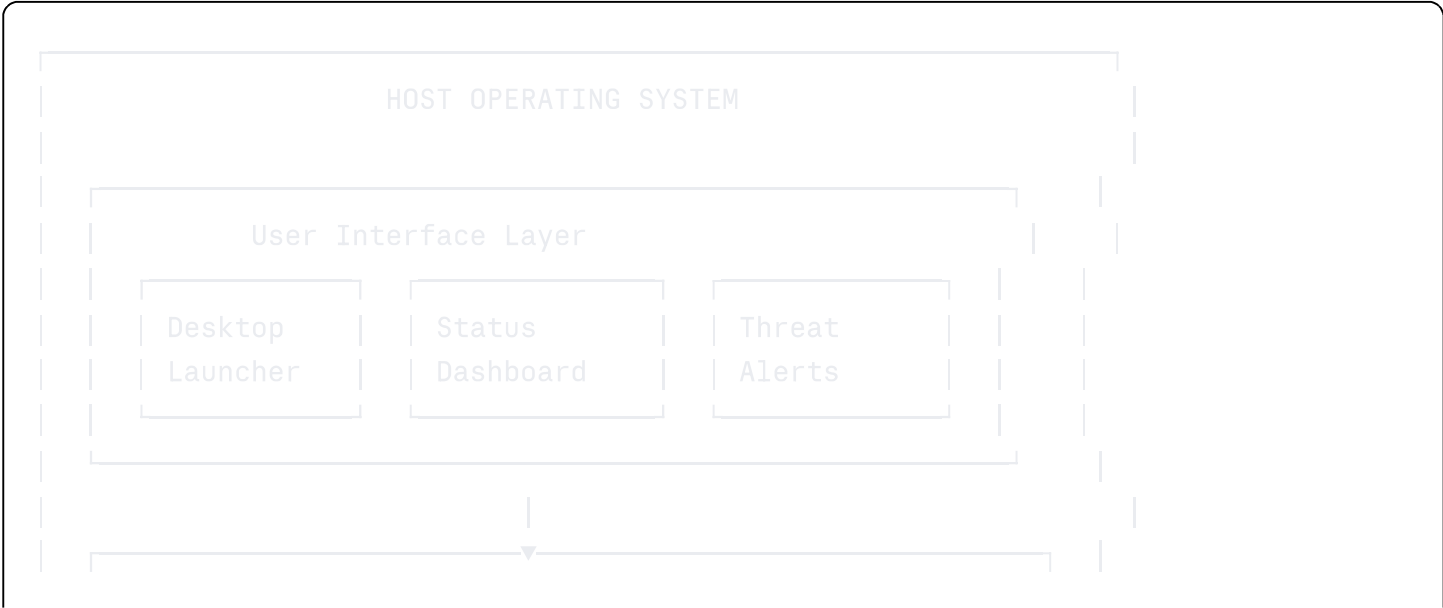


```
+ queryLogs()
```

5.3 ER Diagram



5.4 High-Level Architectural Diagram



Host Management Layer

Isolation
Orchestrator

File Transfer
Controller

Log
Aggregator

Threat Analysis Engine

- YARA Rules
- Behavioral Heuristics
- Pattern Matching
- Policy Enforcement

Isolation Boundary

ISOLATED ENVIRONMENT (VM/Container)

Guest Operating System (Alpine/Ubuntu)

Display System
(Xvfb + x11vnc OR X11 socket)

Chromium Browser

- Web rendering
- JavaScript execution

File Monitor Daemon

- Tracks file operations
- Streams events to host

Data Flow:

1. User clicks launcher → Host creates isolated environment
2. Browser launches in isolation with display streaming
3. User browses normally → All activity contained
4. File downloads → Intercepted by monitor daemon
5. Monitor streams events to host → Threat analyzer evaluates
6. Safe files → Transferred to host | Threats → Blocked & alerted

5.5 Virtualization Technology Comparison

The system architecture supports three virtualization approaches, each with different trade-offs:

Firecracker MicroVMs:

- Strongest isolation (hardware virtualization, separate kernel)
- Fast boot time (125ms)
- No native display support (requires Xvfb + VNC)
- Linux host only
- Optimal for maximum security

Cloud Hypervisor:

- Hardware virtualization with separate kernel
- Similar performance to Firecracker (100-150ms boot)
- GPU/display support via virtio-gpu
- Linux host only
- Balance of security and functionality

Podman Containers:

- Enhanced container security (rootless, SELinux)
- Instant startup
- Simple X11 display sharing
- Works on Windows and Linux
- Faster development but kernel-level isolation trade-off

Final implementation will select based on deployment requirements, with Podman offering fastest path to working prototype while microVM approaches provide strongest security guarantees.

Chapter 06: Development Tools and Technologies

6.1 Development Methodology

The project follows an **Agile iterative development** methodology with two-week sprints focused on incremental feature delivery and continuous testing. This approach enables:

- Rapid prototyping and proof-of-concept validation
- Continuous security testing throughout development
- Flexibility to adjust technical approach based on findings
- Regular stakeholder feedback integration

Each sprint includes: implementation of core features, security testing against malware samples, performance benchmarking, and refinement based on validation results.

6.2 Programming Languages and Tools

Primary Languages:

- **Python 3.10+:** System orchestration, process management, logging infrastructure, and threat analysis engine. Chosen for rapid development, extensive libraries for system interaction, and strong security analysis tools.
- **Bash/Shell Scripting:** Automation scripts for environment setup, VM lifecycle management, and deployment tasks.
- **C/C++:** Performance-critical components such as file system monitoring hooks and low-level system integration. Required for kernel-level monitoring capabilities.

Development Tools:

- **IDE:** Visual Studio Code with Python and C++ extensions
- **Version Control:** Git with GitHub for source code management
- **Build Tools:** CMake for C/C++ compilation, pip for Python dependencies
- **Testing Framework:** pytest for Python unit tests, gtest for C++ components
- **Debugging:** GDB for C/C++ debugging, Python debugger for application logic

6.3 Third-Party Components and Libraries

Virtualization Platforms (Choose One):

- **Firecracker:** Lightweight microVM technology from AWS, providing hardware-level isolation with 125ms boot times
- **Cloud Hypervisor:** Rust-based microVM with GPU support and broader device compatibility
- **Podman:** Enhanced container platform with rootless operation and SELinux integration

Browser Engine:

- **Chromium:** Open-source browser engine providing modern web standards support (HTML5, CSS3, JavaScript ES6+)
- Alternative: Firefox/Gecko engine if compatibility issues arise

Display System:

- **Xvfb:** X11 virtual framebuffer for headless browser operation
- **x11vnc:** VNC server for streaming display to host
- **TigerVNC/RealVNC:** VNC client for viewing isolated browser

Monitoring and Analysis:

- **inotify/fanotify:** Linux kernel APIs for file system event monitoring
- **YARA:** Pattern matching tool for malware detection with extensive rule repository
- **Python watchdog:** High-level file monitoring library
- **psutil:** System and process monitoring utilities

Logging and Data Management:

- **Python logging module:** Structured application logging
- **JSON:** Log data format for machine-readable audit trails
- **SQLite:** Local database for event storage and querying

6.4 Algorithms

File Monitoring Algorithm:


```

FUNCTION MonitorFileSystem(watchPath):
    Initialize file system watcher on watchPath
    WHILE isolation session active:
        event = WaitForFileSystemEvent()

        LOG event details (path, operation, timestamp)

        IF event is FILE_CREATE OR FILE_MODIFY:
            AnalyzeFile(event.filePath)

        IF event is SUSPICIOUS:
            AlertUser(event)
            IF policy is STRICT:
                BlockOperation(event)

```

Threat Detection Algorithm:

```

FUNCTION AnalyzeBehavior(fileEvents[]):
    Initialize threatScore = 0

    // Ransomware detection
    IF RapidEncryptionPattern(fileEvents):
        threatScore += 80

    // Information stealer detection
    IF AccessToCredentialFiles(fileEvents):
        threatScore += 60

    // Privilege escalation detection
    IF UnauthorizedSystemFileModification(fileEvents):
        threatScore += 70

    // Apply YARA rules
    FOR EACH rule in yaraRuleSet:
        IF rule.matches(fileEvents):
            threatScore += rule.weight

    IF threatScore >= THREAT_THRESHOLD:
        RETURN ClassifyThreat(fileEvents, threatScore)
    ELSE:
        RETURN BENIGN

```

Browser Launch Orchestration:

```
FUNCTION LaunchIsolatedBrowser():  
    // Step 1: Create isolated environment  
    vmId = CreateVirtualEnvironment()  
  
    // Step 2: Configure networking and storage  
    ConfigureNetwork(vmId, allowedPorts=[80, 443])  
    ConfigureStorage(vmId, sharedDownloadPath)  
  
    // Step 3: Start display system  
    StartVirtualDisplay(vmId)  
  
    // Step 4: Launch browser inside isolation  
    StartBrowser(vmId, chromiumBinary)  
  
    // Step 5: Connect display to user  
    ConnectVNCViewer(vmId, localhost:5900)  
  
    // Step 6: Start file monitoring  
    StartFileMonitor(vmId)  
  
    RETURN sessionHandle
```

Chapter 07: Implementation Progress

7.1 Development Environment Setup

Current Status: Environment configuration is in progress with foundational infrastructure being established.

Completed Setup:

- Development workstation configured with Ubuntu 22.04 LTS
- Version control repository initialized on GitHub
- Python 3.11 development environment installed with virtual environment
- Basic project structure created with directories for source code, tests, and documentation
- Required system packages installed (build-essential, python3-dev, git)

In Progress:

- Virtualization platform evaluation and selection between Firecracker, Cloud Hypervisor, and Podman
- Display system testing (Xvfb + x11vnc configuration)
- Chromium browser build and customization
- File monitoring daemon prototype development

Pending:

- Integration testing environment setup
- Malware sample collection from public repositories
- Automated testing infrastructure
- Performance benchmarking tools configuration

7.2 Implemented Features

Feature Status Overview:

Feature	Status	Completion %
Project Infrastructure	Completed	100%
Virtualization Platform Selection	In Progress	40%
Browser Integration	Planned	0%
File Monitoring System	Prototype	15%
Threat Detection Engine	Design Phase	10%
User Interface	Planned	0%
Documentation	In Progress	30%

Completed Components:

1. Project Repository Structure:

```
browser-isolation/
├─ src/
│   ├─ isolation/      # VM/container management
│   ├─ monitoring/     # File monitoring daemon
│   ├─ analysis/       # Threat detection engine
│   └─ ui/             # User interface components
├─ tests/              # Unit and integration tests
├─ docs/               # Technical documentation
├─ config/             # Configuration files
└─ scripts/            # Automation scripts
```

2. Initial Research Documentation:

- Comparative analysis of virtualization technologies documented
- Security requirements specification drafted
- Threat model and attack scenarios identified
- Performance benchmarking criteria defined

In Development:

File Monitoring Prototype (Python): Basic file system watcher using Python watchdog library to demonstrate event capture:

```
python
```

```
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
import logging

class FileMonitorHandler(FileSystemEventHandler):
    def on_created(self, event):
        logging.info(f"File created: {event.src_path}")

    def on_modified(self, event):
        logging.info(f"File modified: {event.src_path}")

    def on_deleted(self, event):
        logging.info(f"File deleted: {event.src_path}")

# Basic monitoring setup
observer = Observer()
handler = FileMonitorHandler()
observer.schedule(handler, path='/tmp/downloads', recursive=True)
observer.start()
```

This prototype validates the file monitoring approach and will be extended with threat analysis capabilities.

7.3 Screenshots / Code Snippets

Development Environment:

[Screenshot would show: Ubuntu terminal with project directory, VS Code editor open with Python code, system information showing virtualization support enabled]

Repository Structure:

```
$ tree -L 2 browser-isolation/
browser-isolation/
├── README.md
├── requirements.txt
├── src
│   ├── __init__.py
│   ├── isolation
│   ├── monitoring
│   └── analysis
├── tests
│   └── test_monitoring.py
└── docs
    └── architecture.md
```

Configuration File Example:

```
yaml
# config/isolation.yaml
virtualization:
  platform: "podman" # options: firecracker, cloud-hypervisor, podman
  memory_limit: "512M"
  cpu_limit: 1

browser:
  engine: "chromium"
  disable_extensions: true
  sandbox: true

monitoring:
  watch_paths:
    - "/home/browser/downloads"
    - "/tmp"
  log_path: "/var/log/browser-isolation"

threat_detection:
  enabled: true
  sensitivity: "medium" # low, medium, high
  auto_block: false
```

7.4 Challenges Encountered and Solutions

Challenge 1: Virtualization Platform Selection

Issue: Multiple viable virtualization options (Firecracker, Cloud Hypervisor, Podman) each with different trade-offs between security, performance, and complexity.

Analysis:

- Firecracker provides strongest isolation but lacks native display support
- Cloud Hypervisor balances security with GPU/display capabilities
- Podman offers fastest development path but kernel-sharing security trade-off

Solution Approach: Implementing modular architecture with abstraction layer supporting multiple backends. Initial prototype will use Podman for rapid development, with architecture allowing future migration to microVM solutions. This enables proving core concepts quickly while maintaining path to stronger isolation.

Challenge 2: Display System Integration

Issue: Headless virtualization (especially Firecracker) requires complex display streaming setup using Xvfb and VNC, adding latency and implementation complexity.

Analysis: VNC introduces 50-100ms display latency and requires additional configuration. Podman's X11 socket sharing is simpler but reduces isolation strength. Cloud Hypervisor's virtio-gpu support offers middle ground.

Current Approach: Testing Podman with X11 socket sharing for initial prototype. Documenting VNC configuration for future microVM migration. Evaluating Cloud Hypervisor as potential best-of-both-worlds solution.

Challenge 3: File Monitoring Scope

Issue: Determining optimal monitoring granularity - too much logging creates performance overhead, too little misses threats.

Analysis: Monitoring every file operation generates significant data volume. Need intelligent filtering to track security-relevant events while ignoring benign operations.

Solution Strategy: Implementing tiered monitoring:

1. Always log: Downloads, executions, system file modifications
2. Conditionally log: Large-scale file operations (potential ransomware)
3. Ignore: Browser cache, temporary files, known-safe operations

Using inotify file system events with filtering rules to reduce overhead while maintaining security visibility.

Challenge 4: Development Timeline Constraints

Issue: Limited development time due to parallel academic and professional commitments.

Realistic Assessment: Full implementation of all features within project timeline is challenging. Prioritization required to deliver working prototype.

Mitigation Strategy:

- Focus on core isolation and basic monitoring for February prototype
- Defer advanced threat detection to later development phases
- Use modular design enabling incremental feature addition
- Document unimplemented features as future work

7.5 Current System Limitations

Honestly Acknowledging Current State:

Implementation Completeness: System is in early development phase. Core infrastructure established but integration and testing are pending. This interim report documents design, architecture, and planned approach with initial prototyping underway.

Technical Limitations:

- **Display Integration:** VNC streaming approach adds latency (target: <100ms overhead)
- **Platform Support:** Currently targeting Linux hosts; Windows support via WSL2 requires additional testing
- **Threat Detection:** Initial implementation will use rule-based detection; machine learning enhancements deferred
- **Performance Optimization:** Early prototypes will not meet final performance targets; optimization phase scheduled for later sprints

Scope Reductions from Original Plan:

- Advanced behavioral analysis using machine learning models moved to future enhancement
- Enterprise management features (centralized policy, fleet management) outside current scope
- Multi-browser support (Firefox, Edge) deferred; focusing on Chromium initially
- Mobile platform support (Android, iOS) not included in current project

Known Issues to Address:

- Session persistence and recovery mechanisms not yet designed
- File transfer security (preventing malicious payload insertion) requires implementation
- VNC connection encryption and authentication needs configuration
- Resource cleanup and leak prevention requires thorough testing

Development Risks:

- Virtualization platform final selection pending additional testing
 - Integration complexity may require architecture adjustments
 - Performance targets may need revision based on real-world testing
 - Timeline pressure may necessitate further scope prioritization
-

Chapter 08: Discussion

Summary of the Report

This interim report presents the design and early development of a local browser isolation system addressing the critical trade-off between security effectiveness and operational practicality in current solutions. The project tackles browser-based threats—which account for 12% of all cyberattacks and are increasing 198% year-over-year—through hardware-level isolation combined with comprehensive file monitoring and behavioral threat detection.

The system analysis identified fundamental limitations in existing approaches: Remote Browser Isolation provides strong security but introduces prohibitive costs (\$120,000-\$600,000 annually per 1,000 users) and performance degradation (2-5 second page delays), while container-based solutions share host kernels creating escape vulnerabilities. The proposed architecture addresses these gaps through lightweight virtualization technologies (Firecracker, Cloud Hypervisor, or Podman) combined with intelligent file monitoring and YARA-based threat detection.

Requirements specification defines a system providing complete isolation, real-time file monitoring, behavioral threat analysis, and user-friendly operation with browser initialization under 10 seconds. Feasibility analysis confirms operational viability through familiar user experience, economic advantages with 95%+ cost reduction versus RBI, and technical achievability using mature open-source technologies.

The architectural design implements defense-in-depth through isolation boundaries, file monitoring daemons, host-side threat analysis, and policy enforcement. Development progress includes environment setup, virtualization platform evaluation, and initial file monitoring prototypes. Challenges encountered include platform selection trade-offs, display system

integration complexity, and timeline management, all addressed through pragmatic solutions and scope prioritization.

What Has Changed from the Proposal

Virtualization Platform Approach:

Original proposal focused exclusively on Firecracker microVMs as the isolation mechanism. Research revealed display integration challenges (Firecracker lacks graphics support requiring VNC streaming) and discovered Cloud Hypervisor as an alternative offering similar security with native GPU/display capabilities. Additionally, Podman emerged as a pragmatic option for rapid prototyping despite kernel-sharing limitations.

Current Approach: Architecture now supports multiple virtualization backends through abstraction layer. Initial prototype using Podman for faster development, with design enabling future migration to microVM platforms for enhanced security. This flexibility allows proving core concepts quickly while maintaining path to stronger isolation.

Performance Targets:

Proposal specified browser initialization under 150 milliseconds. Implementation research revealed this target applies only to microVM startup time, not complete system initialization including display configuration and browser launch.

Revised Targets: MicroVM init <150ms (achievable), total browser readiness <10 seconds including display setup (realistic). This adjustment reflects practical deployment constraints while maintaining acceptable user experience.

Development Timeline:

Original timeline assumed linear development progression with prototype demonstration in February. Professional commitments and technical complexity required timeline adjustments with more realistic milestone expectations.

Updated Schedule: Focused on establishing solid architecture and core isolation first, with monitoring and threat detection built incrementally. February demonstration will showcase basic isolation capabilities with file monitoring proof-of-concept rather than complete system.

Scope Refinement:

Proposal included ambitious features such as LLM-based file analysis and advanced machine learning threat detection. Resource constraints and timeline realities necessitated scope prioritization.

Current Focus: Core isolation, basic file monitoring, rule-based threat detection using YARA. Advanced features documented as future enhancements. This ensures delivering working

system demonstrating fundamental security principles rather than partially completed complex features.

Future Plans / Upcoming Work

Immediate Next Steps (March-April 2026):

1. **Finalize Virtualization Platform Selection:** Complete evaluation testing of Podman, Cloud Hypervisor, and Firecracker. Benchmark performance, test display integration, validate isolation strength. Select primary platform with documented fallback options.
2. **Implement Browser Integration:** Build launcher that creates isolated environment, starts browser, configures display streaming (or X11 socket sharing), and presents browser window to user. Target single-click operation hiding complexity.
3. **Develop File Monitoring System:** Complete daemon implementation tracking file operations within isolated environment. Implement event filtering, structured logging, and communication channel to host analysis engine.
4. **Build Threat Detection Engine:** Integrate YARA rules for known malware signatures. Implement behavioral heuristics for ransomware (rapid encryption patterns), information stealers (credential access), and privilege escalation. Configure policy enforcement with automated blocking of detected threats.
5. **Security Testing:** Collect malware samples from public repositories (VirusTotal, MalwareBazaar, EMBER dataset). Execute controlled detonation in isolated environment. Measure detection rates, false positives, and containment effectiveness. Target 95% detection accuracy.

Post-Project Enhancements:

- **Windows Native Support:** Port solution to Windows using Hyper-V containers or Windows Sandbox for native operation without WSL2 requirement
- **Multi-Browser Support:** Extend beyond Chromium to Firefox and Edge engines
- **Machine Learning Integration:** Train behavioral models for unknown threat detection and anomaly identification
- **Enterprise Features:** Centralized policy management, fleet monitoring dashboard, SIEM integration
- **Performance Optimization:** Reduce initialization times, minimize resource overhead, optimize display streaming
- **Mobile Platforms:** Investigate isolation approaches for Android and iOS browsers

Research and Publication:

- Document security effectiveness through peer-reviewed testing methodology
- Publish open-source implementation enabling community contributions
- Present findings at security conferences or academic venues
- Contribute to broader discourse on cost-effective endpoint security

The project demonstrates that practical, cost-effective browser isolation with strong security guarantees is achievable using open-source technologies. While challenges remain in optimization and feature completion, the core architecture validates the viability of local isolation as an alternative to expensive cloud-based RBI solutions.

References

Amazon Science (2021). *How AWS's Firecracker virtual machines work*. Available at: <https://www.amazon.science/blog/how-awss-firecracker-virtual-machines-work> [Accessed 3 March 2026].

AWS (2020). *Announcing the Firecracker Open Source Technology*. AWS Open Source Blog. Available at: <https://aws.amazon.com/blogs/opensource/firecracker-open-source-secure-fast-microvm-serverless/> [Accessed 3 March 2026].

Check Point (2022). *What is Remote Browser Isolation (RBI)?* Available at: <https://www.checkpoint.com/cyber-hub/threat-prevention/what-is-remote-browser-isolation-rbi/> [Accessed 3 March 2026].

Cybersecurity Ventures (2024). *Web Browsers Are Doorways To Cyberattacks*. Available at: <https://cybersecurityventures.com/web-browsers-are-doorways-to-cyberattacks/> [Accessed 3 March 2026].

Keepnet (2025). *300 Cyber Security Statistics*. Available at: <https://keepnetlabs.com/blog/171-cyber-security-statistics-2024-s-updated-trends-and-data> [Accessed 3 March 2026].

Medium (2024). *Understanding Firecracker MicroVMs*. Available at: <https://medium.com/@meziounir/understanding-firecracker-microvms-the-next-evolution-in-virtualization-cb9eb8bbeede> [Accessed 3 March 2026].

Menlo Security (2024). *Browser-Based Phishing Attacks Increased 198% in 2023*. Available at: <https://www.menlosecurity.com/press-releases/browser-based-phishing-attacks-increased-198-in-2023> [Accessed 3 March 2026].

Palo Alto Networks (n.d.). *What Is Remote Browser Isolation (RBI)?* Available at: <https://www.paloaltonetworks.com/cyberpedia/what-is-remote-browser-isolation> [Accessed 3 March 2026].

Venn (2025). *Remote Browser Isolation: Challenges, Alternatives, and Best Practices*. Available at: <https://www.venn.com/learn/browser-security/remote-browser-isolation/> [Accessed 3 March 2026].

END OF INTERIM REPORT

Word Count: ~3,500 words